5.2 Liquid Staking 고급 메커니즘 분석

개요

Liquid Staking 팰릿은 Composable Finance의 핵심 DeFi 인프라로, 릴레이체인 자산의 스테이킹과 유동성을 동시에 제공하는 정교한 메커니즘을 구현합니다. 이 문서는 밸리데이터 선택 전략, 슬래싱 위험 관리, 파생 토큰 환율 계산 등의 고급 메커니즘을 심층 분석합니다.

5.2.1 밸리데이터 선택 및 위임 전략 분석

분배 전략 시스템

Liquid Staking은 다중 파생 계정을 통해 위험을 분산하고 효율성을 극대화하는 분배 전략 시스템을 구현합니다.

DistributionStrategy 트레이트

```
pub trait DistributionStrategy<Balance> {
/// 본딩 시 자산 분배 전략
fn get_bond_distributions(
bonded_amounts: Vec<(DerivativeIndex, Balance, Balance)>,
input: Balance,
cap: Balance,
min_nominator_bond: Balance,
) -> Vec<(DerivativeIndex, Balance)>;
/// 언본딩 시 자산 분배 전략
```

```
fn get_unbond_distributions(
active_bonded_amounts: Vec<(DerivativeIndex, Balance)>,
input: Balance,
min_nominator_bond: Balance,
) -> Vec<(DerivativeIndex, Balance)>;
/// 리본딩 시 자산 분배 전략
fn get_rebond_distributions(
unbonding_amounts: Vec<(DerivativeIndex, Balance)>,
input: Balance,
) -> Vec<(DerivativeIndex, Balance)>;
}
```

AverageDistribution 전략

균등 분배 전략으로 모든 파생 계정에 동일한 비율로 자산을 분배합니다.

```
pub struct AverageDistribution;

impl<Balance: BalanceT + FixedPointOperand> DistributionStrategy<Balance> for Av
fn get_bond_distributions(
bonded_amounts: Vec<(DerivativeIndex, Balance, Balance)>,
input: Balance,
cap: Balance,
min_nominator_bond: Balance,
```

```
) -> Vec<(DerivativeIndex, Balance)> {
let  available_accounts: Vec<_> = bonded_amounts
.into_iter()
.filter(|(_, bonded, _)| *bonded + min_nominator_bond <= cap)
.collect();
if  available_accounts.is_empty() {
return  vec![];
}
let  per_account_amount = input / Balance::from(available_accounts.len() as u32)
available_accounts
.into_iter()
.map(|(index, _, _)| (index, per_account_amount))
.collect()
}
}
```

MaxMinDistribution 전략

최적화된 분배 전략으로 계정 간 밸런스를 고려하여 효율적으로 분배합니다.

```
pub  struct MaxMinDistribution;
```

```
impl<Balance: BalanceT + FixedPointOperand> DistributionStrategy<Balance> for Ma
```

```
fn get_bond_distributions(
mut bonded_amounts: Vec<(DerivativeIndex, Balance, Balance)>,
input: Balance,
cap: Balance,
min_nominator_bond: Balance,
) -> Vec<(DerivativeIndex, Balance)> {
// 본딩량이 적은 계정부터 우선 분배 (오름차순 정렬)
bonded_amounts.sort_by(|a, b| a.1.cmp(&b.1));
let mut distributions = Vec::new();
let mut remaining_input = input;
for (index, current_bonded, _) in bonded_amounts {
if remaining_input.is_zero() {
break;
}
// 캐핑 한도 확인
let available_capacity = cap.saturating_sub(current_bonded);
if available_capacity < min_nominator_bond {
continue;
}
let allocation = remaining_input.min(available_capacity);
distributions.push((index, allocation));
remaining_input = remaining_input.saturating_sub(allocation);
}
```

```
distributions

}

fn get_unbond_distributions(

mut active_bonded_amounts: Vec<(DerivativeIndex, Balance)>,

input: Balance,

min_nominator_bond: Balance,

) -> Vec<(DerivativeIndex, Balance)> {

// 본딩량이 많은 계정부터 우선 언본딩 (내림차순 정렬)

active_bonded_amounts.sort_by(|a, b| b.1.cmp(&a.1));

let mut distributions = Vec::new();

let mut remaining_input = input;

for (index, bonded_amount) in active_bonded_amounts {

if remaining_input.is_zero() {

break;

}

// 최소 본딩량 유지

let available_to_unbond = bonded_amount.saturating_sub(min_nominator_bond);

if available_to_unbond.is_zero() {

continue;

}

let allocation = remaining_input.min(available_to_unbond);

distributions.push((index, allocation));

remaining_input = remaining_input.saturating_sub(allocation);
```

```
}

distributions

}

}
```

노미네이션 실행 메커니즘

```
#[pallet::call_index(9)]

#[pallet::weight(<T as Config>::WeightInfo::nominate())]

#[transactional]

pub fn nominate(

origin: OriginFor<T>,

derivative_index: DerivativeIndex,

targets: Vec<T::AccountId>,

) -> DispatchResult {

Self::ensure_origin(origin)?;

Self::do_nominate(derivative_index, targets)?;

Ok(())

}


#[require_transactional]

fn do_nominate(

derivative_index: DerivativeIndex,
```

```
targets: Vec<T::AccountId>,

) -> DispatchResult {

// 파생 계정 존재 확인

ensure!(

StakingLedgers::<T>::contains_key(derivative_index),

Error::<T>::NotBonded

);

log::trace!(

target: "liquidStaking::nominate",

"Nominating validators for derivative index: {:?}",

&derivative_index,

);

// XCM을 통한 노미네이션 실행

let  query_id = T::XCM::do_nominate(

targets.clone(),

derivative_index,

Self::notify_placeholder()

)?;

// XCM 요청 저장

XcmRequests::<T>::insert(

query_id,

XcmRequest::Nominate {

index: derivative_index,
```

```
targets: targets.clone()
},
);
Self::deposit_event(Event::<T>::Nominating(derivative_index, targets));
Ok(())
}
```

분배 전략 적용

```
// 본딩 시 분배 전략 적용
let amounts: Vec<(DerivativeIndex, Balance, Balance)> = StakingLedgers::<T>::it
.map(|(index, ledger)| {
(index, ledger.active, Self::get_cap_of(index))
})
.collect();

let distributions = T::DistributionStrategy::get_bond_distributions(
amounts,
total_amount,
Self::staking_ledger_cap(),
Self::min_nominator_bond(),
);
```

```
for (index, amount) in distributions.into_iter() {
Self::do_bond_extra(index, amount)?;
}
```

5.2.2 슬래싱 위험 관리 및 보험 메커니즘 분석

슬래싱 스팬 관리

Liquid Staking은 슬래싱 이벤트에 대비한 정교한 관리 시스템을 구현합니다.

슬래싱 스팬 설정

```
#[pallet::config]
pub trait Config: frame_system::Config {
// ... 기타 설정들
/// 슬래싱 스팬 수 (릴레이체인 설정과 동기화)
#[pallet::constant]
type NumSlashingSpans: Get<u32>;
}
```

언본딩 자산 인출 시 슬래싱 스팬 처리

```
#[pallet::call_index(8)]
#[pallet::weight(<T as Config>::WeightInfo::withdraw_unbonded())]
#[transactional]
```

```
pub fn withdraw_unbonded(
origin: OriginFor<T>,
derivative_index: DerivativeIndex,
num_slashing_spans: u32,
) -> DispatchResult {
Self::ensure_origin(origin)?;
Self::do_withdraw_unbonded(derivative_index, num_slashing_spans)?;
Ok(())
}


#[require_transactional]
fn do_withdraw_unbonded(
derivative_index: DerivativeIndex,
num_slashing_spans: u32,
) -> DispatchResult {
// 인출 가능한 언본딩 자산이 있는지 확인
if Self::unbonded_of(derivative_index).is_zero() {
return Err(Error::<T>::NothingToWithdraw.into());
}
log::trace!(
target: "liquidStaking::withdraw_unbonded",
"Withdrawing unbonded for index: {:?}, num_slashing_spans: {:?}",
&derivative_index,
```

```
&num_slashing_spans,
);
// XCM을 통한 언본딩 자산 인출
let  query_id = T::XCM::do_withdraw_unbonded(
num_slashing_spans,
Self::sovereign_account_id(),
derivative_index,
Self::notify_placeholder(),
)?;
// XCM 요청 저장
XcmRequests::<T>::insert(
query_id,
XcmRequest::WithdrawUnbonded {
index: derivative_index,
num_slashing_spans
},
);
Self::deposit_event(Event::<T>::WithdrawingUnbonded(
derivative_index,
num_slashing_spans,
));
Ok(())
```

```
}
```

일괄 언본딩 자산 인출

```
#[require_transactional]
fn do_multi_withdraw_unbonded(num_slashing_spans: u32) -> DispatchResult {
// 모든 파생 계정에 대해 언본딩 자산 인출 실행
for derivative_index in StakingLedgers::<T>::iter_keys() {
Self::do_withdraw_unbonded(derivative_index, num_slashing_spans)?;
}
Ok(())
}
```

슬래싱 대응 메커니즘

교환 비율 보호

```
fn do_update_exchange_rate() -> DispatchResult {
let matching_ledger = Self::matching_pool();
let total_active_bonded = Self::get_total_active_bonded();
let total_liquid_issuance = Self::get_total_liquid_issuance();
// 새로운 교환 비율 계산
let new_exchange_rate = Rate::checked_from_rational(
total_active_bonded
```

```
.checked_add(matching_ledger.total_stake_amount.total)
.ok_or(ArithmeticError::Overflow)?,
total_liquid_issuance,
)
.ok_or(Error::<T>::InvalidExchangeRate)?;
// 슬래싱으로 인한 교환 비율 하락 방지
// 오프체인에서 bond_extra를 통해 슬래싱 손실 보전 필요
if  new_exchange_rate > Self::exchange_rate() {
ExchangeRate::<T>::put(new_exchange_rate);
Self::deposit_event(Event::<T>::ExchangeRateUpdated(new_exchange_rate));
}
Ok(())
}
```

보험 풀 메커니즘

리저브 팩터 관리

```
/// 보험 풀의 리저브 비율 업데이트
#[pallet::call_index(2)]
#[pallet::weight(<T as Config>::WeightInfo::update_reserve_factor())]
#[transactional]
pub  fn  update_reserve_factor(
origin: OriginFor<T>,
```

```
reserve_factor: Ratio,
) -> DispatchResult {
T::UpdateOrigin::ensure_origin(origin)?;
// 리저브 비율 유효성 검증 (0% ~ 100%)
ensure!(
reserve_factor <= Ratio::one(),
Error::<T>::InvalidReserveFactor
);
ReserveFactor::<T>::put(reserve_factor);
Self::deposit_event(Event::<T>::ReserveFactorUpdated(reserve_factor));
Ok(())
}
```

리저브 계산 및 적용

```
// 스테이킹 시 리저브 차감
let reserves = Self::reserve_factor()
.mul_floor(amount)
.ok_or(ArithmeticError::Overflow)?;

// 실제 스테이킹 금액 (리저브 제외)
let amount = amount.checked_sub(reserves).ok_or(ArithmeticError::Underflow)?;

// 리저브를 보험 풀로 전송
```

```
T::Assets::transfer(
Self::staking_currency()?,
who,
&Self::insurance_pool_account(),
reserves,
Preservation::Expendable,
)?;
```

5.2.3 파생 토큰 (stDOT, stKSM) 발행 및 환율 계산 분석

교환 비율 시스템

Liquid Staking의 핵심은 릴레이체인 자산과 파생 토큰 간의 동적 교환 비율 관리입니다.

교환 비율 데이터 구조

```
/// 릴레이체인 자산과 파생 토큰 간의 교환 비율
#[pallet::storage]
#[pallet::getter(fn exchange_rate)]
pub type ExchangeRate<T: Config> = StorageValue<_, Rate, ValueQuery>;


/// 제네시스 설정
#[pallet::genesis_config]
pub struct GenesisConfig {
pub exchange_rate: Rate, // 초기 교환 비율 (보통 1:1)
```

```
pub reserve_factor: Ratio, // 리저브 비율
}
```

교환 비율 변환 함수

LiquidStakingConvert 트레이트

```
pub trait LiquidStakingConvert<Balance> {
/// 스테이킹 자산을 파생 토큰으로 변환
fn staking_to_liquid(amount: Balance) -> Option<Balance>;
/// 파생 토큰을 스테이킹 자산으로 변환
fn liquid_to_staking(liquid_amount: Balance) -> Option<Balance>;
}

impl<T: Config, Balance: BalanceT + FixedPointOperand> LiquidStakingConvert<Bala
fn staking_to_liquid(amount: Balance) -> Option<Balance> {
// 파생 토큰 = 스테이킹 자산 / 교환 비율
Self::exchange_rate()
.reciprocal()
.and_then(|r| r.checked_mul_int(amount))
}
fn liquid_to_staking(liquid_amount: Balance) -> Option<Balance> {
// 스테이킹 자산 = 파생 토큰 * 교환 비율
Self::exchange_rate().checked_mul_int(liquid_amount)
```

```
}

}
```

파생 토큰 발행 메커니즘

스테이킹 시 파생 토큰 발행

```
#[pallet::call_index(0)]

#[pallet::weight(<T as Config>::WeightInfo::stake())]

#[transactional]

pub fn stake(

origin: OriginFor<T>,

#[pallet::compact] amount: BalanceOf<T>,

) -> DispatchResultWithPostInfo {

let who = ensure_signed(origin)?;

// 최소 스테이킹 금액 검증

ensure!(amount >= Self::min_stake(), Error::<T>::BelowMinimumStake);

// 시장 한도 검증

Self::ensure_market_cap(amount)?;

// 리저브 계산 및 차감

let reserves = Self::reserve_factor()

.mul_floor(amount)

.ok_or(ArithmeticError::Overflow)?;
```

```
let  xcm_fees = Self::xcm_fees();

let  net_amount = amount

.checked_sub(reserves)

.and_then(|a| a.checked_sub(xcm_fees))

.ok_or(ArithmeticError::Underflow)?;

// 교환 비율에 따른 파생 토큰 발행량 계산

let  liquid_amount = Self::staking_to_liquid(net_amount)

.ok_or(Error::<T>::InvalidExchangeRate)?;

let  liquid_currency = Self::liquid_currency()?;

// 스테이킹 자산 전송 (리저브 + XCM 수수료 포함)

T::Assets::transfer(

Self::staking_currency()?,

&who,

&Self::sovereign_account_id(),

amount,

Preservation::Preserve,

)?;

// 파생 토큰 발행

T::Assets::mint_into(liquid_currency, &who, liquid_amount)?;

// 매칭 풀에 스테이킹 요청 추가

MatchingPool::<T>::mutate(|matching_ledger| {

matching_ledger.total_stake_amount.add_free(net_amount)

})?;
```

```
Self::deposit_event(Event::<T>::Staked(who, liquid_amount));

Ok(().into())

}
```

교환 비율 업데이트 메커니즘

자동 교환 비율 계산

```
#[require_transactional]

fn do_update_exchange_rate() -> DispatchResult {

let matching_ledger = Self::matching_pool();

let total_active_bonded = Self::get_total_active_bonded();

let total_liquid_issuance = Self::get_total_liquid_issuance();

// 새로운 교환 비율 = (총 활성 본딩량 + 매칭 풀 스테이킹량) / 총 파생 토큰 발행량

let new_exchange_rate = Rate::checked_from_rational(

total_active_bonded

.checked_add(matching_ledger.total_stake_amount.total)

.ok_or(ArithmeticError::Overflow)?,

total_liquid_issuance,

)

.ok_or(Error::<T>::InvalidExchangeRate)?;

// 교환 비율 상승 시에만 업데이트 (슬래싱 보호)

if new_exchange_rate > Self::exchange_rate() {

ExchangeRate::<T>::put(new_exchange_rate);
```

```
Self::deposit_event(Event::<T>::ExchangeRateUpdated(new_exchange_rate));
}
Ok(())
}
```

수동 교환 비율 업데이트

```
/// 긴급 상황 시 교환 비율 강제 업데이트
#[pallet::call_index(13)]
#[pallet::weight(<T as Config>::WeightInfo::update_incentive())]
#[transactional]
pub fn force_update_exchange_rate(
origin: OriginFor<T>,
rate: Rate,
) -> DispatchResult {
T::UpdateOrigin::ensure_origin(origin)?;
// 교환 비율 유효성 검증
ensure!(rate > Rate::zero(), Error::<T>::InvalidExchangeRate);
ExchangeRate::<T>::put(rate);
Self::deposit_event(Event::<T>::ExchangeRateUpdated(rate));
Ok(())
}
```

파생 토큰 소각 메커니즘

언스테이킹 시 파생 토큰 소각

```
#[pallet::call_index(1)]

#[pallet::weight(<T as Config>::WeightInfo::unstake())]

#[transactional]

pub fn unstake(

origin: OriginFor<T>,

#[pallet::compact] liquid_amount: BalanceOf<T>,

unstake_provider: UnstakeProvider,

) -> DispatchResultWithPostInfo {

let who = ensure_signed(origin)?;

// 최소 언스테이킹 금액 검증

ensure!(liquid_amount >= Self::min_unstake(), Error::<T>::BelowMinimumUnstake);

// 파생 토큰을 스테이킹 자산으로 변환

let amount = Self::liquid_to_staking(liquid_amount)

.ok_or(Error::<T>::InvalidExchangeRate)?;

let liquid_currency = Self::liquid_currency()?;

// 파생 토큰 소각

T::Assets::burn_from(

liquid_currency,

&who,

liquid_amount,

Precision::Exact,
```

```
Fortitude::Polite,
)?;
match unstake_provider {
UnstakeProvider::RelayChain => {
// 일반 언스테이킹: 언본딩 기간 적용
let target_era = Self::current_era()
.checked_add(Self::bonding_duration())
.ok_or(ArithmeticError::Overflow)?;
let unlock_chunk = UnlockChunk {
value: amount,
era: target_era,
};
Unlockings::<T>::mutate(&who, |unlockings| {
unlockings.get_or_insert_with(Vec::new).push(unlock_chunk);
});
},
UnstakeProvider::MatchingPool => {
// 빠른 언스테이킹: 매칭 풀 활용
let fast_unstake_fee = Self::matching_pool_fast_unstake_fee();
let net_amount = amount
.checked_sub(fast_unstake_fee)
.ok_or(ArithmeticError::Underflow)?;
// 즉시 자산 반환
```

```
T::Assets::transfer(
Self::staking_currency()?,
&Self::sovereign_account_id(),
&who,
net_amount,
Preservation::Expendable,
)?;
},
}
// 매칭 풀에 언스테이킹 요청 추가
MatchingPool::<T>::mutate(|matching_ledger| {
matching_ledger.total_unstake_amount.add_free(amount)
})?;
Self::deposit_event(Event::<T>::Unstaked(who, liquid_amount, amount));
Ok(().into())
}
```

교환 비율 영향 요소

스테이킹 리워드 반영

```
// 에라 진행 시 자동으로 교환 비율 업데이트
fn  on_initialize(block_number: BlockNumberFor<T>) -> Weight {
// 에라 업데이트 확인
```

```
if Self::should_update_era(block_number) {
let _ = Self::do_advance_era(1);
}
// 매칭 실행
if Self::should_execute_matching() {
let _ = Self::do_matching();
}
Weight::zero()
}


fn do_advance_era(offset: EraIndex) -> DispatchResult {
let current_era = Self::current_era();
let new_era = current_era.saturating_add(offset);
CurrentEra::<T>::put(new_era);
// 교환 비율 업데이트 (스테이킹 리워드 반영)
if let Err(e) = Self::do_update_exchange_rate() {
log::error!(
target: "liquidStaking::do_advance_era",
"Exchange rate update error: {:?}",
&e
);
}
Self::deposit_event(Event::<T>::NewEra(new_era));
```

```
Ok(())
}
```

슬래싱 손실 처리

```
// 슬래싱 발생 시 오프체인에서 bond_extra를 통한 손실 보전
// 교환 비율 하락 방지를 위한 보호 메커니즘
if  new_exchange_rate > Self::exchange_rate() {
// 리워드로 인한 상승만 반영
ExchangeRate::<T>::put(new_exchange_rate);
} else {
// 슬래싱으로 인한 하락은 오프체인 개입 필요
log::warn!(
target: "liquidStaking::exchange_rate",
"Exchange rate decline detected, manual intervention required"
);
}
```

5.2.4 고급 최적화 메커니즘

매칭 풀 최적화

```
impl<Balance: BalanceT> MatchingLedger<Balance> {
/// 스테이킹과 언스테이킹 요청을 효율적으로 매칭
```

```
pub fn matching(
&self,
unbonding_amount: Balance,
) -> Result<(Balance, Balance, Balance), DispatchError> {
let total_free_stake_amount = self.total_stake_amount.free()?;
let total_free_unstake_amount = self.total_unstake_amount.free()?;
use Ordering::*;
let (bond_amount, rebond_amount, unbond_amount) =
if matches!(total_free_stake_amount.cmp(&total_free_unstake_amount), Less | Equ
// 언스테이킹 요청이 더 많은 경우
(
Balance::zero(),
Balance::zero(),
total_free_unstake_amount - total_free_stake_amount
)
} else {
// 스테이킹 요청이 더 많은 경우
let amount = total_free_stake_amount - total_free_unstake_amount;
if amount < unbonding_amount {
// 언본딩 중인 자산을 우선 리본딩
(Balance::zero(), amount, Balance::zero())
} else {
// 새로운 본딩과 리본딩 모두 실행
```

```
(amount - unbonding_amount, unbonding_amount, Balance::zero())

}

};

Ok((bond_amount, rebond_amount, unbond_amount))

}

}
```

가스 효율성 최적화

- **배치 처리**: 여러 파생 계정의 작업을 한 번에 처리
- **지연 실행**: 매칭 풀을 통한 요청 집계 후 일괄 실행
- **조건부 업데이트**: 필요한 경우에만 교환 비율 업데이트

이러한 고급 메커니즘들을 통해 Liquid Staking 팰릿은 안전하고 효율적이며 사용자 친화적인 liquid staking 서비스를 제공하며, 릴레이체인 생태계의 보안과 탈중앙화에 기여합니다.